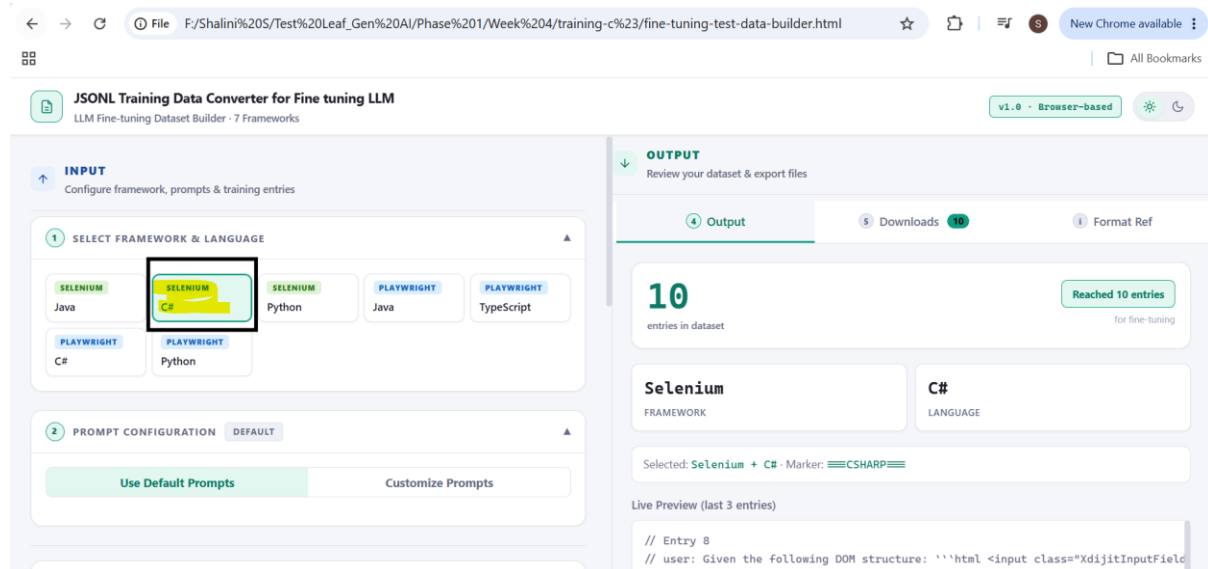


Classroom Assignment

1. Convert HTML Files in the JSONL Format

Select Input (Framework & Training Entries) as C#, Selenium. Upload the 10(Min) files and Click on Process. Then download the JSONL File.

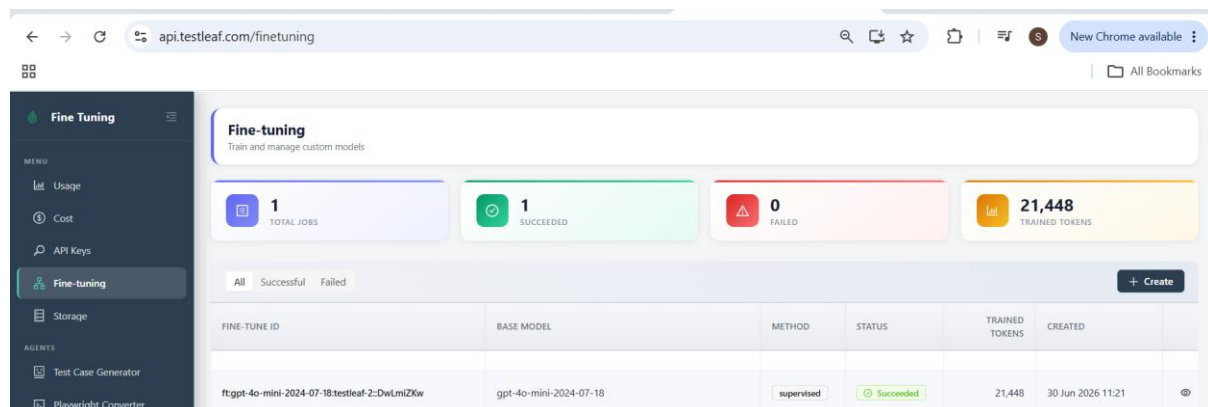


2. Create Fine Tuning Model

In <https://api.testleaf.com/> - Storage > Upload (File that we download in the JSONL Format from the step 1). Status will be ready

3. Finetuning Process

Click on Fine Tuning > Create. Select Method as Supervised, Base Model from LOV, Select Existing, Hyperparameters – Batch Size as 1, Learning Rate Multiplier as 0.3, Number of Epochs as 4. Click on Create Model



4. Click on API Key from Menu > Create, Enter the name and click on Create Key – Name as **API_Fine Tune1** & Key as **n3o0+5EGXXIz**

The screenshot displays the Testleaf CRM/SA application interface. The main section is the 'Create Lead' form, which includes fields for Company Name, Forename, Source, First Name (Local), Salutation, Title, Annual Revenue, Industry, Ownership, SIC Code, Description, Parent Account, Surname, Marketing Campaign, Last Name (Local), Date of birth, Department, Preferred Currency, Number Of Employees, Ticker Symbol, and Important Note. Below the form is a 'Contact Information' section with fields for Country Code, Area Code, Extension, E-Mail Address, Phone Number, Person to Ask For, Web Uri, Primary Address, Attention Name, Address Line 2, State/Province, City, Postal Code, and Zip/Postal Code Extension. A 'Create Lead' button is at the bottom of the form.

On the right side, there is an AI Browser Extension for Code Generation sidebar. It shows the 'Code Generator' settings, including 'Supported Language' (C#), 'Browser Engine' (Selenium), 'Select LLM Provider' (Testleaf), 'Select Model' (API_Fine Tune1), and 'Testleaf API Key'. Below the settings, there is a 'Reset Chat' button and a code editor showing the following code:

```
package com.testleaf.pages;

import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;

/**
 * Page Object for CreateLeadPage
 */
public class CreateLeadPage {

    @FindBy(id = "createLeadForm_compa
```

1. Explore new tool and programming language combinations in the HTML application shared by Sudarshan. For example, such as Playwright with JavaScript, Playwright with TypeScript, Selenium with Java, etc. After exploring the combinations, generate the corresponding JSONL file.

The screenshot shows the 'JSONL Training Data Converter for Fine tuning LLM' application. The interface is divided into two main sections: 'INPUT' and 'OUTPUT'.

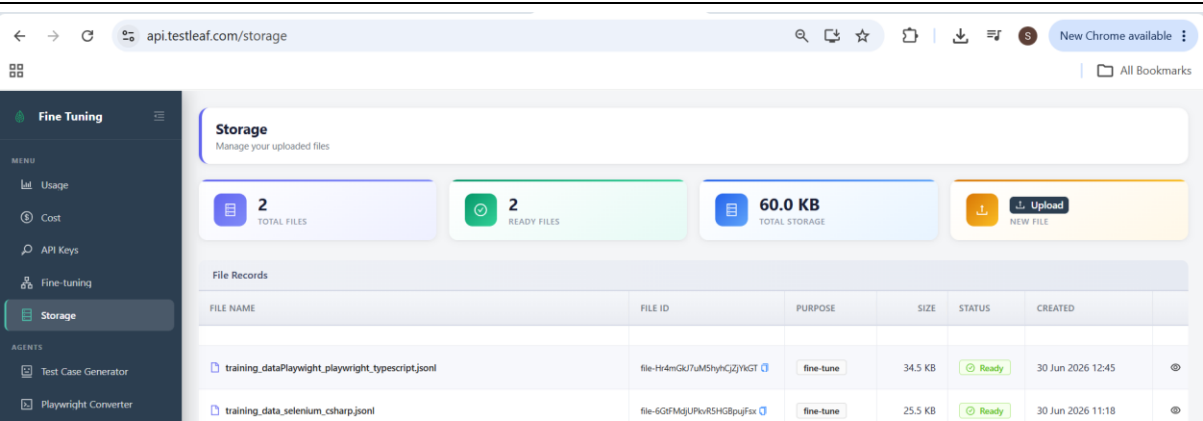
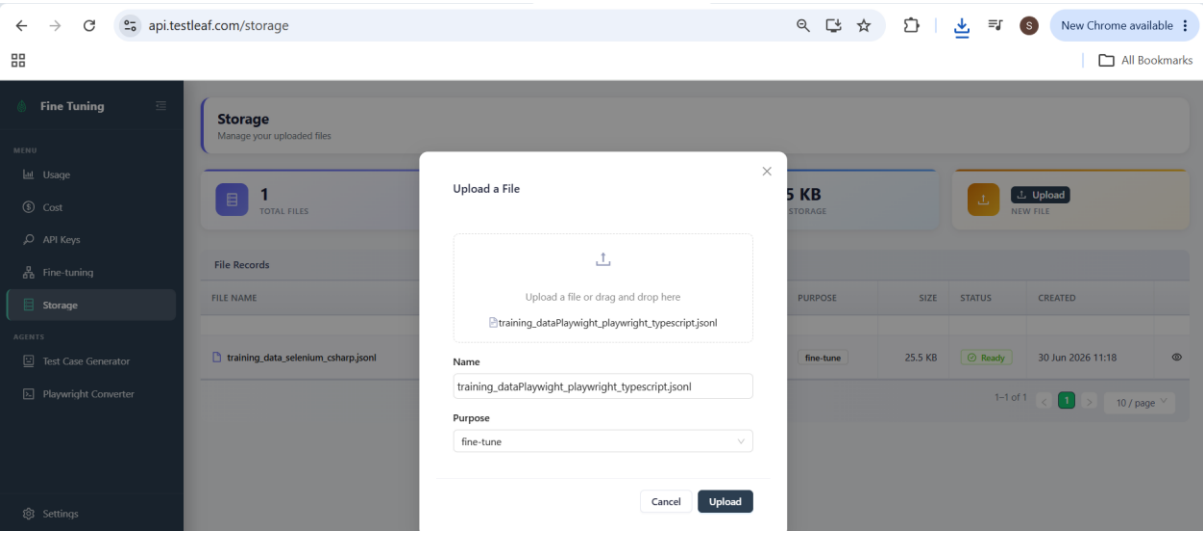
The 'INPUT' section is titled 'Configure framework, prompts & training entries' and includes three sub-sections:

- 1. SELECT FRAMEWORK & LANGUAGE:** This section allows users to select a framework (Selenium or Playwright) and a language (Java, C#, Python, or TypeScript). The 'PLAYWRIGHT' and 'TypeScript' options are currently selected.
- 2. PROMPT CONFIGURATION:** This section includes a 'Use Default Prompts' button and a 'Customize Prompts' button.
- 3. ADD TRAINING DATA:** This section includes a '+ Single Entry' button and a 'Batch Upload' button.

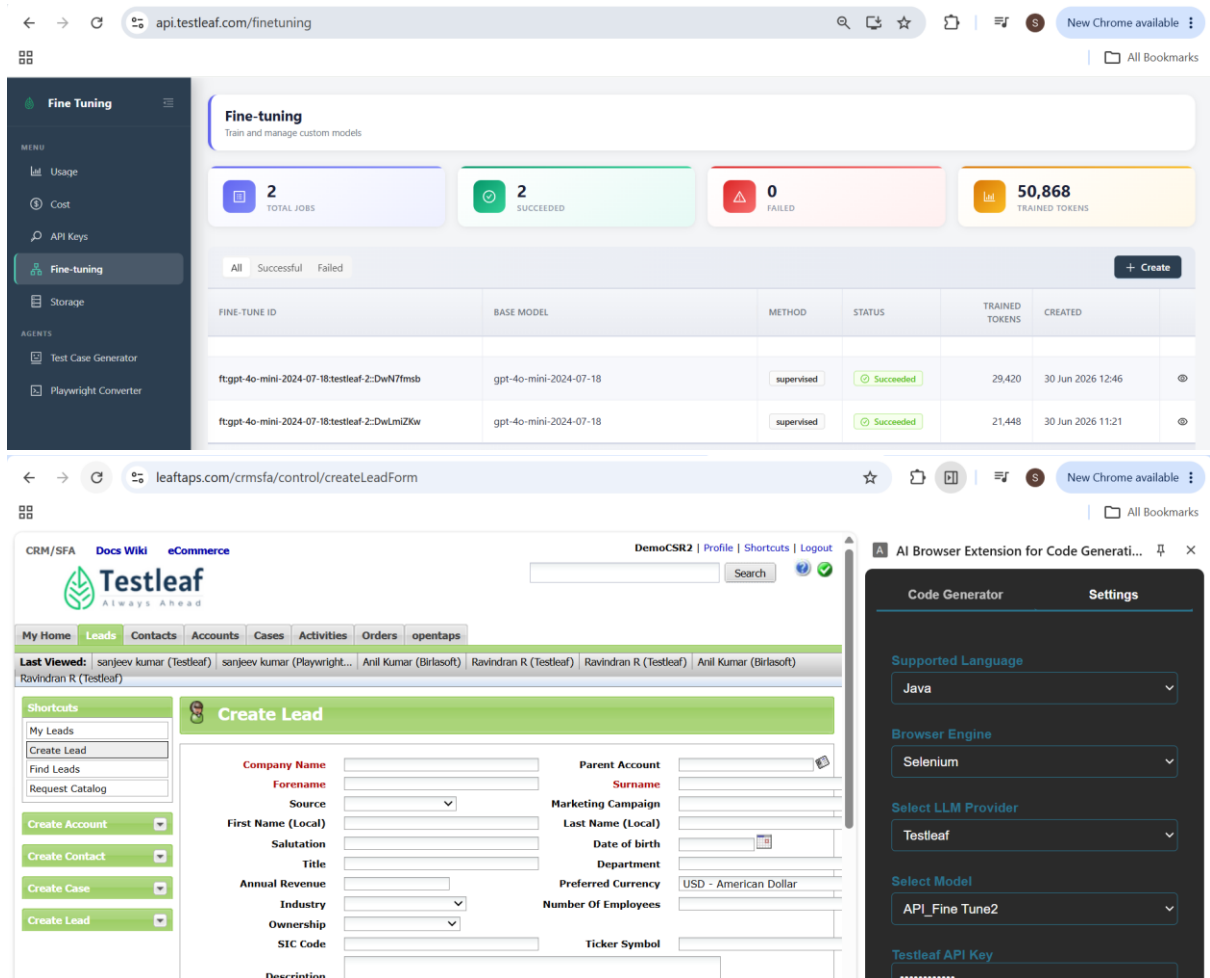
The 'OUTPUT' section is titled 'Review your dataset & export files' and includes three sub-sections:

- Output:** This section shows the 'OUTPUT CONFIGURATION' with fields for 'Output filename' (training_dataPlaywright), 'Extension' (.jsonl), and 'Auto-download batch' (Off).
- Downloads:** This section shows the 'PER-ENTRY DOWNLOADS' and a 'DOWNLOAD FULL DATASET' button. The filename for the full dataset is 'training_dataPlaywright_playwright_typescript.jsonl'.
- Format Ref:** This section shows the 'Download All as JSONL' button.

Trying for **Playwright with JavaScript**



File ID - file-Hr4mGkJ7uM5hyhCjZjYkGT



package com.testleaf.pages;

```
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.WebElement;
```

```
import org.openqa.selenium.support.FindBy;
```

```
import org.openqa.selenium.support.PageFactory;
```

```
/**
```

```
 * Page Object for Lead Creation Page
```

```
 */
```

```
public class LeadCreationPage {
```

```
    private WebDriver driver;
```

```
@FindBy(id = "createLeadForm_companyName")
```

```
private WebElement companyNameInput;
```

```
@FindBy(id = "createLeadForm_firstName")
```

```
private WebElement firstNameInput;
```

```
@FindBy(id = "ext-gen521")
```

```
private WebElement createLeadButton;
```

```
/**
```

```
 * Constructor to initialize the Page Object with the WebDriver instance.
```

```
 *
```

```
 * @param driver the WebDriver instance
```

```
 */
```

```
public LeadCreationPage(WebDriver driver) {
```

```
    this.driver = driver;
```

```
    PageFactory.initElements(driver, this);
```

```
}
```

```
/**
```

```
 * Set the company name in the company name input field.
```

```
 *
```

```
 * @param companyName the company name to set
```

```
 */
```

```
public void enterCompanyName(String companyName) {
```

```
    companyNameInput.clear();
```

```
    companyNameInput.sendKeys(companyName);
```

```
}
```

```
/**
```

* Set the first name in the first name input field.

*

* @param firstName the first name to set

*/

```
public void enterFirstName(String firstName) {
```

```
    firstNameInput.clear();
```

```
    firstNameInput.sendKeys(firstName);
```

```
}
```

```
/**
```

* Click the create lead button to submit the form.

*/

```
public void clickCreateLeadButton() {
```

```
    createLeadButton.click();
```

```
}
```

```
}
```